

Coursework Overview

The goal is to practice complete form handling: creating an HTML form, submitting to a PHP endpoint, validating user input, and writing records to a server file. This index now contains the core reference content directly so it can be used as a standalone worksheet.

This is a CW 03 in-class group assignment. You must work with your teammate listed in the course spreadsheet and submit one complete team deliverable with visible contributions from both members.

Original objective: practice writing a PHP script that processes an HTML form and perform server-side validation. The activity theme is a joke "Buy a Grade" form that sends data to a PHP script.

How To Use This Page

Build while reading. For each exercise: copy the template, run a test, inspect output, and then explain what changed and why. The reflection prompts at the end are part of the assignment deliverable.

Mission Board: Learn By Building

Mission 1: Observe

Identify what each input field sends to PHP and predict the resulting `$_POST` keys.

Mission 2: Repair

Fix broken data flow such as multi-select posting, then verify with controlled test inputs.

Mission 3: Persist

Store clean records to a file and inspect how each request modifies persistent state.

Mission 4: Protect

Reject incomplete submissions and provide clear next-step guidance to users.

- ☐ I tested a valid form submission and verified all fields.
- ☐ I tested at least one invalid submission case.
- ☐ I confirmed records append correctly to `suckers.html`.
- ☐ I can explain why `options[]` is required in multi-select forms.

Asset Strategy: Connect Files To Objectives

Use each asset intentionally. Do not treat files as separate tasks; treat them as parts of one data pipeline.

Asset	Role In Development	Objective It Supports
<code>buyagrade.html</code>	User input collection layer	Build correct form controls and parameter names
<code>sucker.php</code>	Request processing layer	Read, validate, and render submitted data
<code>suckers.html</code>	Flat-file storage layer	Persist structured records using append mode
<code>index.css</code>	Learning UI and readability	Improve comprehension and workflow navigation
<code>Bug_issue.png</code>	Debug scenario evidence	Diagnose form-to-PHP mismatch in posted values
<code>ex1-expected-output.png</code> to <code>ex4-expected-output.png</code>	Acceptance checkpoints	Compare your output against expected behavior
<code>formtest.zip</code>	Optional extension practice	Transfer debugging pattern to another form

Exercise 1: Debugging Challenge

Problem to solve: only one value from the multi-select list is submitted. Fix the form and PHP processing so all selected features are captured and displayed.

- Test with 2 or 3 selected options: Chat, Email, File transfer, Directory, Shell, Remote control.
- Submit random values and verify all selected options are printed by PHP.

► Hint: Why only one option appears

Debug Target: Multi-Select Form Fix

```
<form action="debug-form-handler.php" method="post">
  <label>Your Name: <input name="visitor_name" type="text"></label>
  <label>Password: <input name="password" type="password"></label>

  <!-- Important fix: name must end with [] for multi-select arrays -->
  <label>Optional features:</label>
  <select name="options[]" size="6" multiple>
    <option>Chat</option>
    <option>Email</option>
    <option>File transfer</option>
    <option>Directory</option>
    <option>Shell</option>
    <option>Remote control</option>
  </select>

  <button type="submit">Sign Up</button>
</form>
```

Debug Target: PHP Array Output Fix

```
<?php
echo '<h1>Form input values</h1>';
echo '<p>Your Name: ' . htmlspecialchars($_POST['visitor_name'] ?? '') .

$options = $_POST['options'] ?? [];
if (!is_array($options)) {
    $options = [$options];
}

echo '<p>Options: ' . htmlspecialchars(implode(', ', $options)) . '</p>'

echo '<h2>All Form Data</h2>';
echo '<pre>';
```

```
print_r($_POST);
echo '</pre>';
?>
```

Exercise 2: Create the Form

Build the complete `buyagrade.html` form. This reflects the reference structure from the exercise examples and keeps field names expected by the PHP script.

► Simple guide

Reference Form (Completed)

```
<form action="sucker.php" method="post">
  <dl>
    <dt>Name</dt>
    <dd><input type="text" name="name"></dd>

    <dt>Section</dt>
    <dd>
      <select name="section">
        <option value="">(Select a section)</option>
        <option>MA</option><option>MB</option><option>MC</option><option>
        <option>ME</option><option>MF</option><option>MG</option><option>
      </select>
    </dd>

    <dt>Credit Card</dt>
    <dd>
      <input type="text" name="cardnumber"><br>
      <label><input type="radio" name="cardtype" value="visa">Visa</labe
      <label><input type="radio" name="cardtype" value="mastercard">Mast
    </dd>
  </dl>
  <div><input type="submit" value="I am a giant sucker."></div>
</form>
```

Exercise 3: Display Input Data in PHP

Create a confirmation page that prints user values and includes raw posted data for debugging while developing.

► **Hint: Safe output**

Reference Processor: Show Submitted Data

```
<h1>Raw Form Data</h1>
```

```
<pre><?php print_r($_POST); ?></pre>
```

```
<h1>Form input values</h1>
```

```
<p>Your Name: <?= htmlspecialchars($_POST['name']) ?></p>
```

```
<p>Section: <?= htmlspecialchars($_POST['section']) ?></p>
```

```
<p>Card Number: <?= htmlspecialchars($_POST['cardnumber']) ?></p>
```

```
<p>Card Type: <?= htmlspecialchars($_POST['cardtype']) ?></p>
```

Exercise 4: Save Form Data

Append each submission to `suckers.html` in this format:

`Name;Section;CardNumber;CardType`.

► Simple guide

Reference Save Logic

```
<?php
$name = trim($_POST['name'] ?? '');
$section = trim($_POST['section'] ?? '');
$cardnumber = trim($_POST['cardnumber'] ?? '');
$cardtype = trim($_POST['cardtype'] ?? '');

$line = $name . ';' . $section . ';' . $cardnumber . ';' . $cardtype . PHP_EOL;
file_put_contents('suckers.html', $line, FILE_APPEND);

$all = file_get_contents('suckers.html');
echo '<h2>The current database contains:</h2>';
echo '<pre>' . htmlspecialchars($all) . '</pre>';
?>
```

Sample Output Records

```
Tiger Woods;MA;1234123412341234;visa
Hulk Hogan;MF;5963109385987345;mastercard
Matt Ryan;MC;7328904328904902;mastercard
Louis Henry;MC;4444100020003000;visa
```

Exercise 5: Basic Validation

Validate that no required field is blank. If input is incomplete, show an error message and stop processing.

► **Hint: Validation order matters**

Reference Validation Block

```
<?php
$required = ['name', 'section', 'cardnumber', 'cardtype'];

foreach ($required as $field) {
    if (!isset($_POST[$field]) || trim($_POST[$field]) === '') {
        echo '<h1>Sorry</h1>';
        echo '<p>You did not fill out the form completely. <a href="buya
        exit;
    }
}
?>
```

Lab Workflow

1. Start from Exercise 1 and test the fixed multi-select behavior.
2. Build the completed Buy-a-Grade form from Exercise 2.
3. Verify incoming values in PHP with `$_POST` and echo them in HTML.
4. Save each successful submission using `file_put_contents(..., FILE_APPEND)`.
5. Re-test with empty fields to confirm validation behavior.
6. Coordinate tasks with your teammate from the spreadsheet list and keep commit evidence for both members.

Success target: this single page contains all needed instructions and code references.

Grading Rubric (100 points)

- **Layout & Responsiveness (20 pts)** - Grid/Flexbox implementation; works at all breakpoints.
- **Accessibility (15 pts)** - Contrast, focus management, reduced motion support.
- **Visual Design & Theming (15 pts)** - Color palette, typography, imagery cohesion.
- **Interactivity (20 pts)** - CSS-only navigation/state via advanced selectors.
- **Animation Quality (15 pts)** - Smooth, purposeful, performance-friendly animations.
- **Code Organization (10 pts)** - Clean, commented CSS; semantic HTML structure.
- **Collaboration & Submission (5 pts)** - Visible contributions from both members; complete deliverables.

Submission Template

Complete this page and submit it with your project.

Team Information

Fill in your team details using the same naming format as your GitHub commits.

- Team Member: First N: Last N:
- Team Member: First N: Last N:

Project Links

Paste your GitHub repository URL and your live CODD deployment URL below.

Copyable Submission Template

Template

Complete this page and submit with your project.

Team Information

Fill in your team details.

Team Member Name

Team Member: First N: Last N:

Team Member: First N: Last N:

Tip: Use the same naming format as your GitHub commits.

Project Links

Paste your GitHub repository URL and your live CODD deployment URL below

GitHub Repository URL

Both team members must have visible commits in this repo.

<https://github.com/>

CODD Live Deployment URL

Your finished project hosted on codd.cs.gsu.edu.

<https://codd.cs.gsu.edu/>

Tip: Open each link in a new tab to verify it works before submitting.

Due Date: 4:15 PM on 07/01/2026

Critical Thinking Questions (Word Document Required)

Students must answer these questions in a Word document and submit it with their code. Focus on your design decisions, debugging process, and evidence from testing.

1. How does changing a form field `name` attribute impact PHP processing and output?
2. Why is `options[]` required for multi-select, and what failure appears without it?
3. What is the difference between checking `isset($_POST['x'])` and checking if a value is non-empty?
4. Why should validation execute before file write operations? What bug does this prevent?
5. What are the trade-offs of storing records in a flat file versus a database for this assignment?
6. How did you verify that your output matches the expected behavior in all five exercises?
7. What changes would you make to improve user trust and form security in a real production system?
8. How could CSS and page structure reduce student mistakes during development?

Copyable Word Document Template

Course: Web Programming

Assignment: CW 03 Forms and PHP

Student Name:

Date:

1) Development Summary

- What I built:

- Files I edited:

- Main objective achieved:

2) Test Evidence

- Test Case A (valid input):

Expected:

Actual:

- Test Case B (blank field):

Expected:

Actual:

- Test Case C (multi-select 2+ options):

Expected:

Actual:

3) Critical Thinking Responses

Q1:

Q2:

Q3:

Q4:

Q5:

Q6:

Q7:

Q8:

4) Reflection

- Biggest debugging challenge:
- What I learned about HTML-PHP data flow:
- What I would improve next:

Expected Output Checkpoints

Checkpoint A: Form submits and displays posted values cleanly.

Checkpoint B: Saved records appear in a `<pre>` block.

Checkpoint C: Blank fields return a clear error and retry link.

Debug reminder: test valid input, empty fields, and multi-select options.